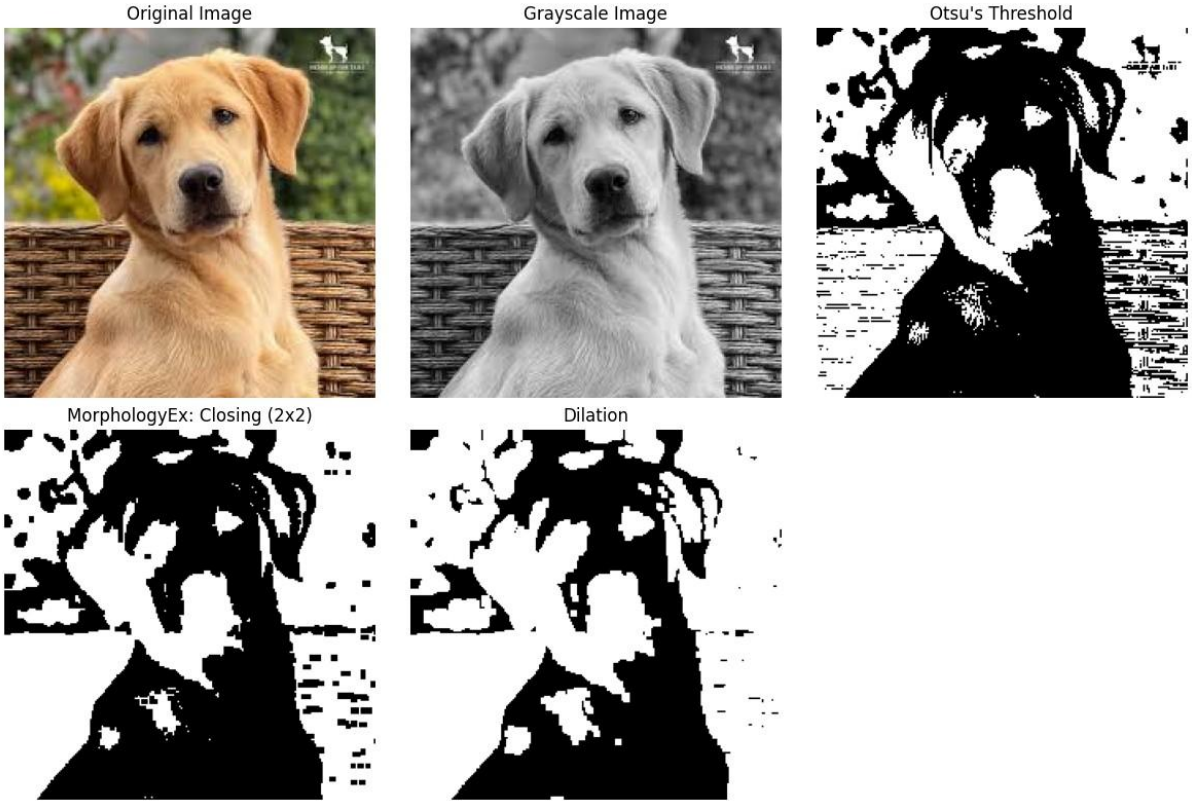
Experiment 16:

Program:

```
import numpy as np import
cv2 from matplotlib import
pyplot as plt from
google.colab import drive

# Mount Google Drive if not already
mounted try:
   drive.mount('/content/drive')
except:
   pass # Drive is already mounted

# For Colab: !wget https://example.com/cataract_image.jpg -O
image.jpg img = cv2.imread('/content/drive/MyDrive/dog.jpeg')

# Check if the image was loaded
successfully if img is None:
   print("Error: Could not load image. Please check the file
path.") else:
   b, g, r = cv2.split(img)     rgb_img = cv2.merge([r, g, b])     gray =
cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)     ret, thresh =
cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
   kernel = np.ones((2, 2), np.uint8)     closing =
cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel,
iterations=2)     sure_bg = cv2.dilate(closing, kernel,
iterations=3)

   plt.figure(figsize=(12, 8))     plt.subplot(231);
plt.imshow(rgb_img); plt.title("Original Image"); plt.axis('off')
plt.subplot(232); plt.imshow(gray, 'gray'); plt.title("Grayscale
Image"); plt.axis('off')     plt.subplot(233); plt.imshow(thresh, 'gray');
plt.title("Otsu's Threshold"); plt.axis('off')     plt.subplot(234);
plt.imshow(closing, 'gray'); plt.title("MorphologyEx: Closing
(2x2)"); plt.axis('off')     plt.subplot(235); plt.imshow(sure_bg,
```

'gray'); plt.title("Dilation"); plt.axis('off') plt.tight_layout()
plt.show()
  plt.imsave('dilation.png', sure_bg)

Output:



Experiment 17:

Program:

import numpy as np import matplotlib.pyplot

as plt from sklearn.datasets import

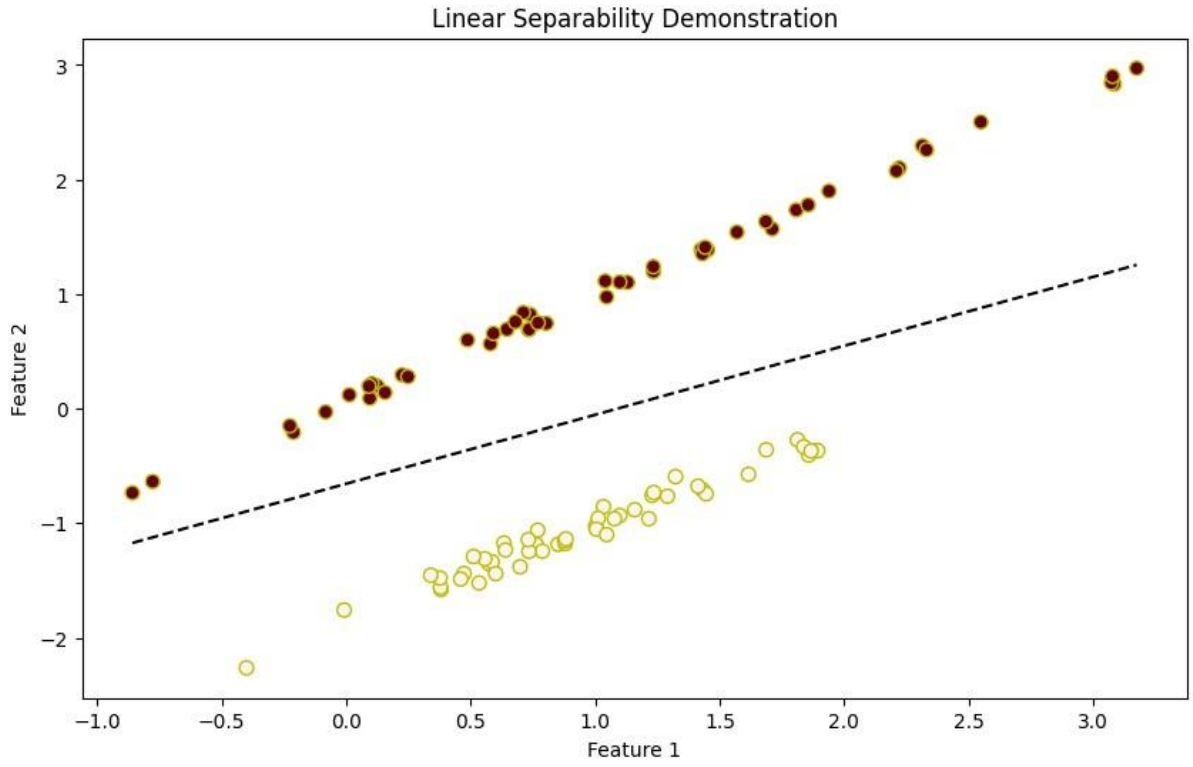
make_classification from sklearn.linear_model

import LogisticRegression from

sklearn.metrics import accuracy_score

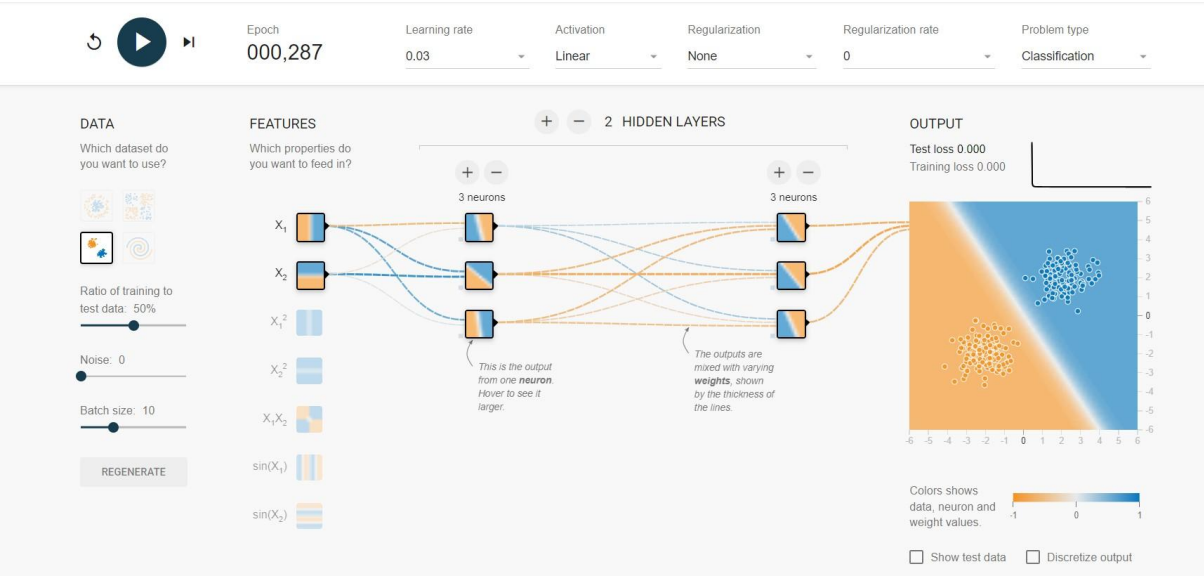
```
X, y = make_classification(n_samples=100, n_features=2,
n_informative=2, n_redundant=0, n_clusters_per_class=1,
random_state=42) model = LogisticRegression()

model.fit(X, y) y_pred = model.predict(X) accuracy =
accuracy_score(y, y_pred) print(f"Accuracy:
{accuracy:.2f}") plt.figure(figsize=(10, 6)) plt.scatter(X[:,
0], X[:, 1], c=y, cmap='berlin', edgecolor='m', s=50) coef =
model.coef_[0] intercept = model.intercept_ x_vals =
np.linspace(X[:, 0].min(), X[:, 0].max(), 100) y_vals = -
(coef[0] * x_vals + intercept) / coef[1] plt.plot(x_vals,
y_vals, 'k--')


plt.xlabel('Feature 1')
plt.ylabel('Feature 2') plt.title('Linear
Separability Demonstration')
plt.show()
i
```

Output:



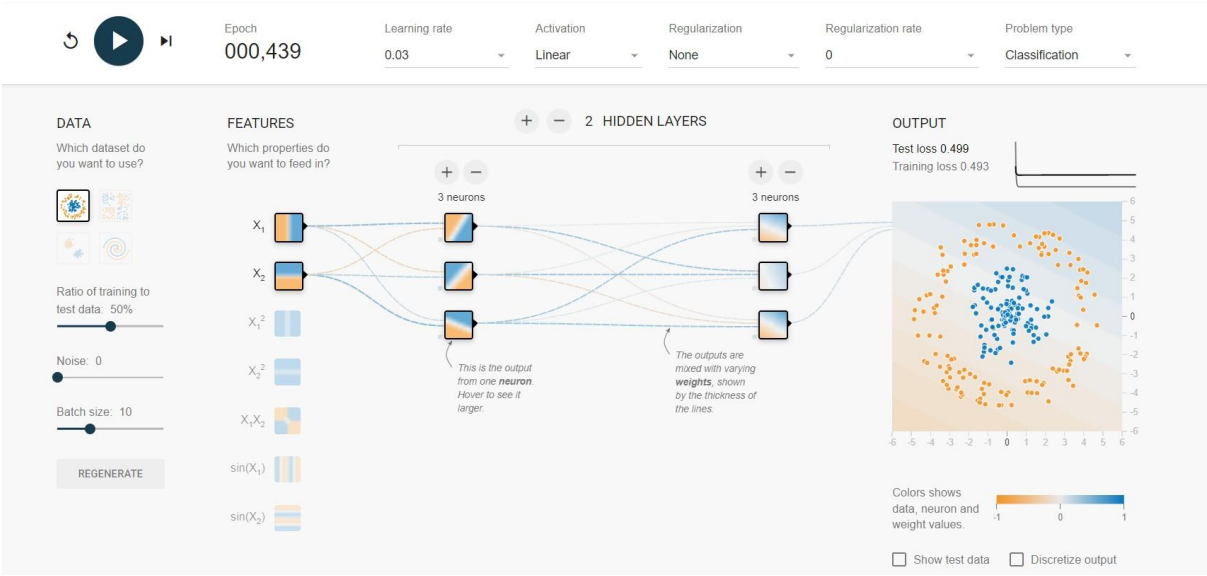
Experiment 18:

Output:



Experiment 19:

Output:



Experiment 20:

Output: